



Low-energy real FFT architectures and their applications to seizure prediction from EEG

Sai Sanjeet¹ · Bibhu Datta Sahoo¹ · Keshab K. Parhi²

Received: 16 February 2022 / Revised: 20 June 2022 / Accepted: 23 August 2022 / Published online: 6 September 2022
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

While many fast Fourier transform (FFT) architectures have been presented for computing real-valued FFT (RFFT), which of these architectures is best suited for low-throughput applications such as bio- medical signals which are typically sampled between 256 Hz and 1 kHz remains unclear. This paper implements and compares throughput, resources, and energy consumption of three different hardware architectures for real-valued FFT algorithms using Xilinx Ultra96-V2 FPGA development board. The RFFT architectures exploit the conjugate symmetry property of the real signals, thereby eliminating about half of the computations compared to a complex FFT. The three FFT architectures investigated in this paper include: single processing element (SPE), pipelined, and *in-place*. It is shown that, for a 256-point RFFT, using FPGA, the *in-place* architectures require the least device resources when compared to the pipelined architectures, while the throughput of the pipelined architectures is approximately 8 times that of the *in-place* architecture. These RFFT architectures are then used to generate feature vectors for a machine-learning based epileptic seizure prediction system. The seizure prediction system using the various RFFT architectures are then realized in Xilinx Ultra96-V2 FPGA development board and the power consumption values of the overall system using these architectures are compared. It is shown that the pipelined implementation of the feature extraction core results in $\approx 30\%$ reduction in power consumption of the entire system than the *in-place* implementation for the same target clock frequency, as the pipelined architecture has a higher throughput and hence is idle for majority of the computation time.

Keywords Fast Fourier Transform (FFT) · Real-Valued FFT · Real-valued signals · Biomedical signals · *in-place* · Pipelined · EEG · Seizure prediction · Feature extraction · Convolutional neural network

1 Introduction

The discrete Fourier transform (DFT) is an important tool for analysis of signals across various fields [1]. The FFT algorithm for computing the DFT significantly reduces the complexity. One of the most widely used FFT algorithm is

the Cooley-Tukey algorithm which recursively breaks down a large FFT into a combination of smaller FFTs by factorization. The Cooley-Tukey algorithm assumes a general signal as the input to the FFT, *i.e.*, the signal is assumed to be complex. But most real-world signals, such as biomedical signals, audio signals, etc., do not contain an imaginary component and their Fourier transforms are conjugate-symmetric. Variants of the Cooley-Tukey algorithm have been proposed for real-valued signals [2–4], which reduce the complexity of the FFT even further. Figure 1 shows the structure of 16-point modified radix-2 *decimation-in-frequency* (DIF) [1] architecture for real-valued signals. The architecture exploits the conjugate-symmetry property of the signal, thereby computing only 9 output samples as opposed to 16.

The FFT is an integral part of most digital signal processing (DSP) based systems that the design of efficient FFT processors is an active area of research. Various methodologies for implementing the FFT processors have been proposed

✉ Bibhu Datta Sahoo
bsahoo@ece.iitkgp.ac.in; bsahoo@ieee.org

Sai Sanjeet
sanjeet029@gmail.com; sanjeet029@kgpian.iitkgp.ac.in

Keshab K. Parhi
parhi@umn.edu

¹ Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology, Kharagpur 721302, India

² Department of Electrical and Computer Engineering, University of Minnesota, Minneapolis, USA

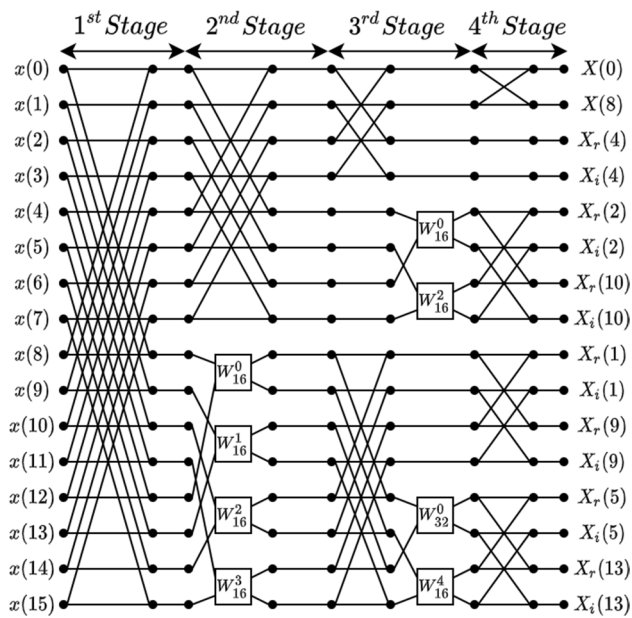


Fig. 1 Structure of modified 16-point radix-2 DIF architecture for real-valued signals ($W_N^M = e^{-j2\pi M/N}$) [2, 3]

[2–8]. All these implementations can be divided into two broad categories: *in-place* architectures, and *pipelined* architectures. The *in-place* architectures utilize the same memory and computational elements for the computation of all the stages [1], whereas *pipelined* architectures have separate memory and compute elements for each stage. This leads to a trade-off between hardware utilization and throughput. The *in-place* architectures requires low hardware area but also operate at a lower throughput, while the *pipelined* architectures achieve a higher throughput but require more hardware resources.

While many FFT architectures have been presented for computing real-valued FFT (RFFT), which of these architectures is best suited for low-throughput applications such as biomedical where the signals are typically sampled between 256 Hz and 1 kHz remains unclear. This paper investigates different RFFT architectures in terms of hardware utilization, power consumption, and throughput for real-valued signals. The three FFT architectures investigated in this paper include: single processing element (SPE), pipelined, and *in-place*. These RFFT architectures are then used to generate feature vectors for a machine learning classifier using convolutional neural networks (CNNs) [9].

A seizure prediction system is taken as the target application to demonstrate the various RFFT architectures. The proposed architectures are implemented and validated on an FPGA as a demonstration of proof of concept. The FPGA

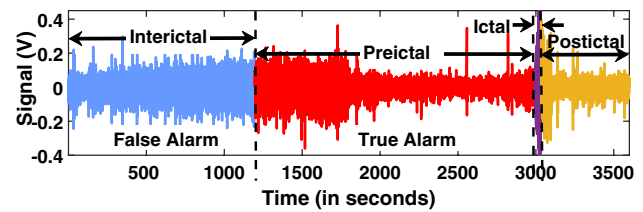


Fig. 2 A typical 1-hour epileptic EEG recording showing various states

board used for validation was Ultra96-V2 development board¹.

2 Seizure prediction: an overview

Epilepsy is a common neurological disease that affects approximately 1% of the world population [11]. Most of the epileptic seizure episodes last less than 2 minutes but can severely interfere with the patients' daily life. Having an early warning system could help in significantly improving the lifestyle of the patient by helping them avoid potentially dangerous tasks such as driving, swimming, etc., and in providing appropriate medication to prevent the forthcoming seizure.

As seizures are a result of abnormal electrical activities in the brain, Electroencephalogram (EEG) is an important tool for detection and prediction of seizures. EEG recordings of seizure patients are divided into four categories:

- *Ictal*: The part of the recording during a seizure.
- *Preictal*: The part of the recording immediately before a seizure.
- *Postictal*: The part of the recording immediately after a seizure.
- *Interictal*: The part of the recording between two seizures, not including Preictal and Postictal.

Figure 2 shows a typical 1-hour epileptic EEG recording showing the four states, viz., *interictal*, *preictal*, *ictal*, and *postictal*. The *ictal* period is only a few seconds, while the *preictal* and *interictal* can span for hundreds of seconds. The goal of an early warning system can be viewed as classifying a period of EEG recording data as *interictal* or *preictal*. As shown in Figure 2, predicting a seizure during *interictal* period leads to a *false-alarm*; the number of such false alarms should be minimized.

¹ <https://www.avnet.com/wps/portal/us/products/new-product-introductions/npi/aes-ultra96-v2/>

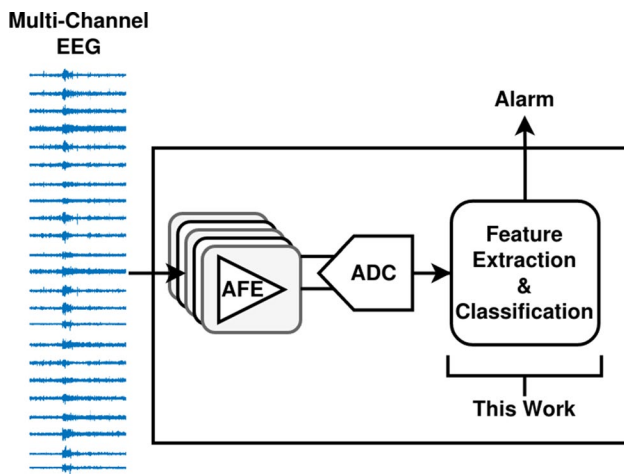


Fig. 3 A typical EEG Seizure Prediction System with an analog data acquisition front-end and a digital processor for prediction

The warning system should be *always-on* and take inputs from the patient continuously or on a periodic basis, *i.e.*, it should operate in real-time. The prediction algorithms used must accurately identify the occurrence of a seizure well in advance with a very low false alarm rate.

Figure 3 shows a hardware approach to realizing a *wearable* epileptic seizure prediction system. The system incorporates a multi-channel analog front-end (AFE) followed by a single ADC for data acquisition and then a digital prediction system, *i.e.*, feature extraction and classification, which has been trained with the *patient specific* data [12].

Epileptic seizure prediction is done either in software or in hardware. The latter facilitates realization of the seizure prediction system as a *wearable* device. There are two main stages for inference in seizure prediction: feature extraction, and classification of the extracted features into *interictal* or *preictal*. In literature, seizure prediction is widely explored using supervised classification techniques, both in software and on hardware, with a majority of the methods using either time domain or frequency domain based feature extraction from EEG recordings. The proposed real-time seizure prediction system uses frequency content of the EEG signal extracted from its FFT as features for a Convolutional Neural Network based classifier.

2.1 Software based seizure prediction

In [13–16] spectral powers of different frequency subbands were used as the feature for classification using *support vector machine* (SVM) based *machine learning* approaches. Shiao et. al. [17] used spectral powers of different frequency subbands along with cross-channel correlation of signals to measure their similarity. Browne et. al. [18] used eigen-spectra and amplitude features extracted from space-delay

correlation and covariance matrices at several delay scales. Kavehei et. al. [19] used Short-Time Fourier Transform to convert the EEG signal to an image-like matrix to be given to the classifier. Wunsch et. al. [20] used the raw EEG signal and its wavelet-decomposed subbands as the training/testing data for classification without any feature extraction.

Deep Learning algorithms have outperformed traditional methods in various classification tasks such as computer vision, speech recognition, character recognition, etc. Some previous works use deep learning based approaches for classification of the extracted features [19, 20, 21–24]. [19] used a Convolutional Neural Network (CNN) on the extracted features. [20] used a Recurrent Neural Network (RNN) for classification.

Generally, Machine Learning based approaches, *viz.*, SVM, require less computation when compared to Neural Networks, but Neural Network based approaches tend to give better results [25].

2.2 Hardware based seizure prediction

There have been a few works on implementing a seizure prediction system in hardware [12, 26–31]. Most of these works propose an ASIC-implementation of the prediction system. In [12], Verma et. al. propose an SoC which employs a signal acquisition front-end, a spectral analysis based feature extraction, and an SVM based classifier. The system is implemented in 180 nm CMOS with upto 18 channels used for prediction. At 1 V operating voltage, the system consumes 9 μ J per channel. Mohsenin et. al. [27] proposed a system with distinct modules for detection and analysis of seizures. The seizure detection unit is run continuously and the analysis unit is activated once a seizure is detected. This reduces the overall power consumption as detection unit performs rudimentary computations and operates at a lower frequency of 256Hz when compared to the more complex analysis unit, which runs at 1.85kHz. At 1.3V the detection unit dissipates 0.04 μ W and the analysis unit consumes 0.36 μ W when powered on. The system is implemented in 65 nm CMOS and occupies 0.43 mm². Yoo et. al. [29, 30] explored seizure prediction in hardware using linear and non-linear SVMs. Usage of non-linear SVMs increases the power consumption of the prediction system. The system using linear SVM consumes 1.52 μ J/classification, while non-linear SVM consumes 1.83 μ J/classification.

The performance of seizure prediction systems is evaluated mainly with two performance metrics, *viz.*, sensitivity and false prediction rate (FPR). Sensitivity is defined as the percentage of seizures correctly predicted by the model, and FPR is defined as the number of false alarms per hour. The aim of the seizure prediction system is to have high sensitivity while maintaining low FPR.

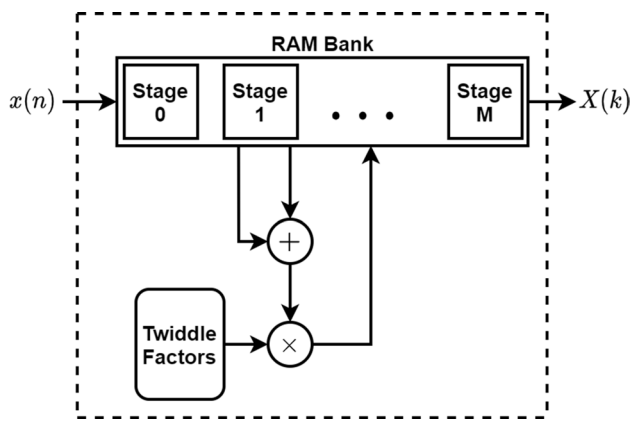


Fig. 4 Block diagram of the SPE FFT architecture for N -point FFT. The multiplier shown here is a complex-multiplier

3 Proposed RFFT architectures

Three different architectures for computing the FFT are proposed [10]. They are

- A single processing element (SPE) based architecture
- Fully-pipelined architecture
- *in-place* FFT architecture.

All the above FFT architectures process inputs and compute outputs in real time. The three architectures use decimation-in-time (DIT) radix-2 algorithm optimized for real-valued signals. In [2], DIF architectures have been described in detail for real-valued signals. However, an interesting feature of these architectures is that they can be easily configured to DIT architecture by just changing the addressing schemes for the input, output, and of the various stages. All the three architectures are explained for an N -point FFT which requires M -stages where $M = \log_2(N)$.

3.1 SPE architecture

We begin with the simple SPE architecture. Figure 4 shows the architecture of the SPE based FFT processor. For an N -point FFT, this architecture consists of $(M + 1)$ random-access memory (RAM) blocks of depth N elements. The RAMs are denoted as Stage-0, Stage-1, ..., and Stage- M . Stage-0 RAM is used to store the input-data whose FFT needs to be computed and the other RAMs are used to store the outputs of the corresponding stage. The PE incorporates an adder and a complex multiplier². The computation in the

butterfly-structure of the FFT happens serially, *i.e.*, output of a given stage is computed after the previous stage has finished its computation, using the same adder and multiplier. Initially, the module is ready to receive inputs and stores them in the input-memory, *i.e.*, stage-0 RAM (see Figure 4), using the *bit-reversed* addressing scheme. The output of the 1st-stage of FFT computation is stored in stage-1 RAM, the output of the 2nd-stage of FFT computation is stored in stage-2 RAM, etc.

Once the input-memory is full, the first stage computation begins. A *location-counter* keeps track of the index of the output to be computed as well as the two inputs, and whether the two inputs are to be added or subtracted. Since the first stage does not involve any multiplication with the twiddle-factors, the input to the multiplier is set to '1'.

Once the first stage computation completes, the second stage computation starts. From the second stage onwards, twiddle-factor multiplication is necessary. As twiddle-factors depend on the FFT stage, a *stage-counter* along with a *location-counter* is used to send the appropriate twiddle-factor to the multiplier. Thus, a *stage-counter* tells which stage computation is being executed and the *location-counter* keeps track of the input to be sent to the PE and where the output needs to be stored. Once the M^{th} stage computation is completed, the output is streamed along with the corresponding frequency index. Since DIT FFT algorithm is used, the frequency index is just the value of the *location-counter* for the corresponding output.

The addressing of inputs by the *location-counter* is a function of the *stage-counter*, and the addressing scheme for the k^{th} stage using DIT will be the same as that of the $(M - k)^{\text{th}}$ stage using DIF. Therefore the two addressing schemes can be multiplexed for every stage and the select line can be used to switch between DIT and DIF implementations of FFT.

3.2 Fully-Pipelined architecture

Figure 5 shows the block level view of the fully-pipelined architecture, based on [3, 4, 7, 8]. The architecture consists of M -stages, and each stage has a set of two RAMs each of depth N . Similar to the SPE architecture, the two input-RAMs store the input data and the other RAMs store the output of each stage. The FFT processor has separate PEs for each stage. The PE in the first-stage uses a real-adder only while the PEs in the subsequent stages use an adder and a complex multiplier.

As opposed to the one memory per stage in the SPE architecture, the fully-pipelined architecture incorporates two memory banks per stage. This is to ensure continuous computation, as one memory stores the output of the previous stage, and the other memory provides input for the computation of the output of next stage, thus facilitating

² A complex multiplier here multiplies two complex numbers over two clock cycles. Thus, the complex multiplier in this case is equivalent to two real multipliers and one real adder.

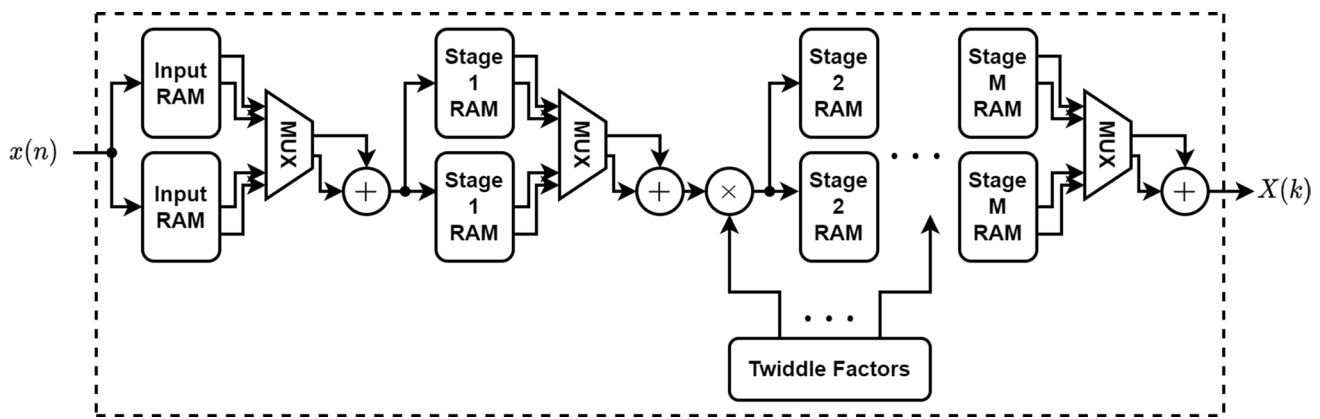


Fig. 5 Block diagram of the fully-pipelined architecture based on [3, 4, 7, 8]. All the multipliers are complex-multipliers, and all the adders are real-adders

real-time FFT computation. The roles of the two memories switch every time the stage finishes its computation. There are no memory elements to store the output of the last stage as the output can be directly streamed along with the corresponding frequency index. Since all the stages compute concurrently, each stage has its own location counter that addresses the input to the PE as well as addresses the output of the PE to the appropriate RAM location. Thus, unlike the SPE architecture, the fully-pipelined architecture does not require a stage-counter. Similar to the SPE architecture, this architecture can be reconfigured on the fly as either DIF FFT or DIT FFT by multiplexing the addressing scheme using a select line.

3.3 In-place FFT architecture

Figure 6 shows the *in-place* FFT architecture based on [2, 5, 6], which is the most hardware-efficient architecture as it uses only two memory banks and one set of computational elements (one real-adder and one complex-multiplier). Each memory bank has a depth of N . For the computation of each stage, one of the memories acts as input RAM while the other memory stores the computed output. The data is cycled only between the two memories present, hence the name *in-place*.

First, the streaming input is stored in the first memory array if the module is ready to accept new data. Once the first memory array is full, the stage 1 computation starts and the outputs are stored in the second memory array. Similar to the SPE architecture, a stage counter keeps track of the stage under computation and appropriate logic is used for addressing the input and storing the output at the correct locations.

Once the computation of the first stage completes, the second memory array acts as input, and the computed outputs are stored in the first memory array. This process repeats until all the M stages are computed, after which the

output is streamed along with the corresponding frequency index. The *in-place* FFT architecture can also be configured on-the-fly as DIF FFT or DIT FFT just like the previous two.

All the additions in all the architectures are pipelined using 2 pipeline registers and all the multiplications with 3 pipeline registers. Since the first and the last stage of the FFT consist of only additions, they require $(N + 2)$ clock cycles. The second stage requires either an addition or a multiplication and therefore takes $(N + 3)$ clock cycles. All the other stages require both an addition and a multiplication and take $(N + 5)$ clock cycles. Therefore the total number of clock cycles for a single FFT is $(M \cdot N + 5(M - 3) + 7)$. For the SPE and *in-place* architectures, the structure of all the stages is the same and therefore requires $(M \cdot (N + 5))$ clock cycles for computation of one FFT. For the SPE and *in-place* architectures, the throughput is the same as their latency as they compute the second FFT after the completion of the first. The pipelined architecture begins the computation of the second FFT after the completion of one stage of

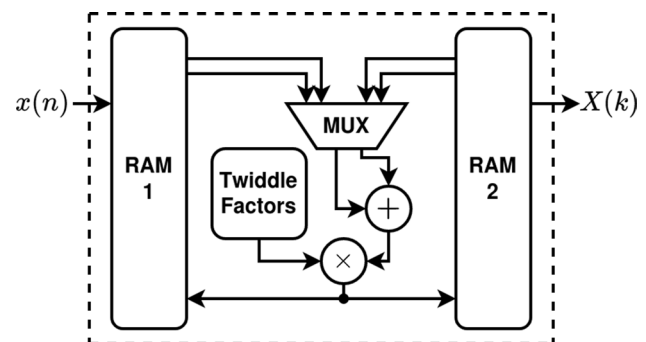


Fig. 6 Block diagram of the *in-place* FFT architecture based on [2, 5, 6]. The adder and multiplier shown here are real-adder and complex-multiplier, respectively

Table 1 Minimum operating clock frequency for an input data rate of 1 kHz

FFT length N	Minimum Operating Frequency (kHz)		
	SPE	Pipelined	In-place
16	5.250	1.313	5.250
64	6.469	1.078	6.469
256	8.156	1.020	8.156
1024	10.050	1.005	10.050
4096	12.015	1.001	12.015

Table 2 Summary of the CHB-MIT Dataset for the patients under consideration

Patient	Number of Seizures	Interictal Hours
chb01	6	14.37
chb02	3	26.04
chb03	7	27.39
chb05	5	14.45
chb06	10	24.80
chb07	3	51.32
chb10	7	24.28
chb14	6	4.71
chb21	4	23.43
chb22	3	17.06
chb23	5	14.22
Total	59	242.08

the first, so its throughput is one FFT per $(N + 5)$ clock cycles. If f_d is the input data rate and f_c is the operating clock frequency of the system, f_c/f_d should be greater than $1/(N \times \text{throughput})$. Therefore the operating clock frequency should be at least $(M + 5 \cdot M/N)$ times the data rate for SPE and *in-place* architecture, and at least $(1 + 5/N)$ times the data rate for the fully-pipelined architecture. Table 1 describes the minimum operating clock frequency, in kHz, of all the three architectures for various FFT lengths when the input data rate is 1 kHz.

4 Seizure prediction system

To demonstrate the RFFT architectures, a frequency spectrum based prediction system that incorporates a CNN for classification is used. The prediction engine processes in N -channel EEG signals of a patient as input. A Fast Fourier Transform (FFT) is computed on the input EEG sequence and its magnitude is computed. The frequency components over 30 seconds are sent to a CNN which classifies the EEG sequence as *interictal* or *preictal*.

4.1 Dataset

The system was trained and tested on the CHB-MIT Scalp EEG Database [32]. The dataset consists of 23 cases from 22 patients with a total of 198 seizures. All the signals are sampled with a sample rate of 256 Hz. A total of 11 patients of the 23 are considered for training and testing the seizure prediction system. The details are tabulated in Table 2.

The part of the signals which are at least four hours away from the nearest seizure are labeled as *interictal*. The duration of the signal after 30 minutes of seizure offset is marked as *postictal*. *Preictal* part of the recording is defined to the segment from 65 minutes to 5 minutes before a seizure onset and not part of any *postictal* duration. Therefore, if there are any seizures which are less than 30 minutes apart, the latter seizure would not be taken into consideration for training or testing.

4.2 Feature extraction

The feature used for classification is the frequency content of the EEG signal. The frequency information is extracted by taking the Short-Time Fourier Transform (STFT) of the signal. A 256-point STFT of the input signal, with an overlap of 50% is computed resulting in a frequency resolution of 1Hz. The feature map for classification is built for a signal duration of 30 seconds, resulting in 59 Fourier Transform output vectors per channel.

The Real-Valued 256-point Fourier Transform generates 129 frequency components, of which the DC component is discarded. In the CHB-MIT Dataset, a power-line noise contaminates the signal at 60Hz and 120Hz. Therefore, frequency components around the power-line frequency (57 – 63Hz) and its harmonic (117 – 123Hz) are removed. This results in a total of 114 frequency components per Fourier transform per channel. Since all the patients under consideration have 23 channels, the shape of the final feature map is $(23 \times 114 \times 59)$.

The architectures described in Section 3 are used to implement the STFT and power consumption and hardware utilization using the fully-pipelined and *in-place* architectures are compared.

4.3 Classifier model

A Convolutional Neural Network (CNN) is trained on the extracted feature map in a patient-specific way. The architecture of the model is tabulated in Table 3.

The model is trained with the *preictal* and *interictal* data of the patients. For every patient, the last 20% of the interictal recordings and the preictal data corresponding to the last 20% of the seizures is used for testing. Since the number of interictal samples are much larger than the number of

Table 3 Network architecture of the CNN

Layer	Kernel Shape	Output Shape
Conv1	(5×5)	$(16 \times 55 \times 28)$
BatchNorm	—	$(16 \times 55 \times 28)$
MaxPool	—	$(16 \times 27 \times 14)$
Conv2	(3×3)	$(32 \times 25 \times 12)$
BatchNorm	—	$(32 \times 25 \times 12)$
MaxPool	—	$(32 \times 12 \times 6)$
Conv3	(3×3)	$(64 \times 10 \times 4)$
BatchNorm	—	$(64 \times 10 \times 4)$
MaxPool	—	$(64 \times 5 \times 2)$
Flatten	—	(640)
FC1	(640×256)	(256)
FC2	(256×2)	(2)

Table 4 Performance of the network for all the patients before and after quantization

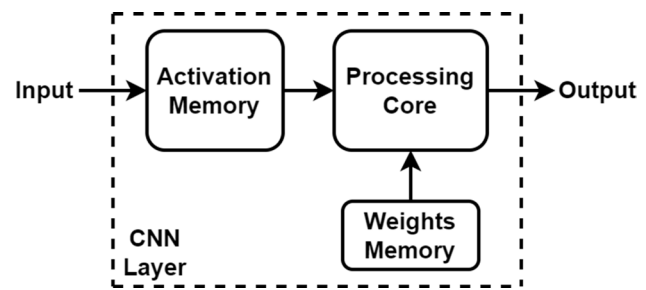
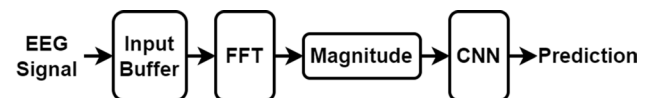
Patient	Floating-Point		Quantized	
	Sen (%)	FPR	Sen (%)	FPR
chb01	92	0.347	92	0.253
chb02	20	0.083	40	0.350
chb03	100	0.589	96.67	0.611
chb05	70	0.167	35	0.167
chb06	76.25	0.842	75	0.899
chb07	80	0.322	70	0.349
chb10	93.33	0.697	53.33	0.620
chb14	72	0.480	76	0.600
chb21	100	0.560	100	0.560
chb22	100	0.875	100	0.850
chb23	100	0	100	0
Total	84.47	0.5	76.60	0.517

preictal samples in most cases, interictal samples are randomly picked for training to match the number of preictal samples. Leave-one-out cross-validation method is used for training and validating the network by placing one seizure in the validation set and using the remaining seizures for training.

The models are first trained with floating-point STFT and floating-point weights to serve as the baseline for comparison. The STFT is then quantized to 16-bit datapaths and the CNN is quantized to 8-bit datapaths. Table 4 shows the performance of the system before and after quantization.

4.4 CNN hardware design

As described in Section 4.2, the input feature map to the CNN is of the size $(23 \times 114 \times 59)$. Storing such a large feature map in memory would be impractical. Therefore,

**Fig. 7** Structure of a layer in the CNN. All the layers follow a similar structure**Fig. 8** Block level view of the Seizure Prediction system

a real-time streaming based CNN architecture is employed with the convolutional layers performing computation only on the parts of the input which are available. All the layers operate in a pipeline in real-time and the output of the CNN is available once every 30 seconds. The structure of all the layers is the same as shown in Figure 7.

Each layer has a processing core with several processing elements which compute a partial dot product every clock cycle. This product is accumulated over multiple clock cycles resulting in the final output of the layer. The number of processing elements in the processing core decides the number of filters that can be computed in parallel and is chosen in a manner that minimum hardware resources are used while still maintaining a real-time operation. Each processing element computes a (5×5) dot product in the first convolutional layer as the filter size is (5×5) for the first layer. All other convolutional layers have filters of size (3×3) and therefore every processing element in the other layers computes a (3×3) dot product every cycle.

The FFT and CNN blocks comprise the main elements of the system, as shown in Figure 8.

5 FPGA implementation of RFFT architectures

The proposed architectures are implemented on an Ultra96-V2 development board³. Verilog is used as a hardware description language and Xilinx Vivado Design Suite-2020.2 is used to perform behavioral and post-synthesis simulations.

³ <https://www.avnet.com/wps/portal/us/products/new-product-introductions/npi/aes-ultra96-v2/>

Table 5 Comparison of the proposed architectures with previous work. The power consumption is reported at a clock frequency of 100 MHz. The throughput is also calculated at 100 MHz

	[3]	[8]	Proposed		
			SPE	Pipelined	<i>In place</i>
FFT Size	256	64	256	256	256
LUTs	3300	3976	3213	1850	223
FFs	3300	3976	247	350	186
BRAMs	0	0	0	8	1.5
DSP Slices	0	0	2	12	2
Power (mW)	18.2	20.8	33	63	11.69
Latency (clock cycles)	–	–	2088	2080	2088
TP [†] (FFT/s)	–	–	47893	383142	47893

[†]Throughput

5.1 RFFT structures

All the architectures described in Section 3 are implemented using a 16-bit datapath. The outputs of the adders and the multipliers are truncated to 16-bits.

The twiddle factors are precomputed and stored as 32-bit complex numbers, of which 16-bits represent the real component and 16-bits represent the imaginary component. The real and imaginary components are in the range $[-1, +1]$. Since a twiddle factor of $+1$ cannot be represented in this notation, a flag pin is incorporated into the complex multipliers which indicates whether the twiddle factor is 1. If the twiddle factor is 1, the input is passed onto the output without any modification.

The major differences between the three architectures are in the way intermediate results are stored in memory.

5.2 SPE implementation

The memories in each stage are implemented as LUTRAMs (distributed memory) as two memory locations need to be accessed per clock cycle. The twiddle factors are also stored in LUTRAMs to facilitate the computation of two different outputs in a stage at the same time. The FPGA utilization details are summarized in Table 5. It is observed that the SPE Implementation is a very naive implementation and is neither power-efficient nor resource-efficient.

5.3 Fully-Pipelined implementation

Unlike the SPE architecture, the datapath in the fully-pipelined architecture is duplicated for the different stages of the FFT. The computation involved in the first-stage uses a single 16-bit adder, while the subsequent stages use one complex-adder and one complex-multiplier. The twiddle

factors are stored in separate memory arrays per stage as Block RAMs (BRAMs).

The fully-pipelined architecture requires almost the same number of clock cycles for the computation of a single FFT as the SPE architecture, but 8 FFTs are computed in the pipeline. Although the latency remains the same⁴, the throughput improves by approximately 8 times. The FPGA utilization details of this architecture are summarized in Table 5. The throughput of the fully-pipelined implementation is 383142 FFTs per second when compared to the 47893 FFTs per second by the SPE architecture, both operating at a clock frequency of 100 MHz.

5.4 In-place implementation

The implementation of the *in-place* architecture also follows the same methodology as the previous two implementations. The structure of the real-adder and the complex-multiplier is similar to that of the ones used in SPE architecture.

The stage counter is used to generate all the control signals for the module, including the select line for selecting the input from the appropriate input, write enable for writing into the correct memory, etc.

The FPGA utilization details are summarized in Table 5. The latency of the *in-place* architecture is similar to that of the SPE architecture, but the *in-place* architecture uses significantly less resource.

Table 5 compares the device utilization and power consumption of the implemented architectures with previous works. It can be observed that the *in-place* architecture is the most resource efficient, while the pipelined architecture consumes less power per FFT.

The number of LUTs and FFs required by both SPE and pipelined architecture for 512-point FFT and 1024-point FFT would be 2.25 times and 5 times, respectively, as that of a 256-point FFT. However, for 512-point and 1024-point FFT using *in-place* architecture the LUTs and FFs would increase, respectively, by 12.5% and 25% with respect to 256-point FFT. In case of pipelined architecture, the number of BRAMs and DSP slices would increase by 12.5% for a 512-point FFT and by 25% for a 1024-point FFT. The number of DSP slices used in the SPE is independent of N . Similarly, the *in-place* architecture always requires 1.5 BRAMs and 2 DSP slices for any N . The latency of all the architectures is proportional to $N \cdot \log_2(N)$. The throughput of both SPE and *in-place* architecture is proportional to $1/(N \cdot \log_2(N))$ while that of pipelined architecture is proportional to $1/N$.

⁴ The fully-pipelined architecture takes 8 fewer clock cycles to compute one FFT due to stage-specific optimizations.

Table 6 Breakup of hardware utilization of the various blocks in the seizure prediction system

Block	Input Buffer	FFT		Mag.	CNN
		Pipe- lined	<i>in-place</i>		
LUTs	142	1011	223	3	3629
FFs	44	1246	186	5	3074
BRAMs	11.5	11	1.5	0	96.5
DSPs	0	12	2	2	43
Power (uW)	1.15	8.03	11.23	0.12	7.93
Latency (clock cycles)	257	7800	53678	4	170, 352

6 FPGA implementation of seizure prediction system

The prediction system, as discussed in Section 4, requires computation of an STFT and a CNN. The input to the system is a 23-channel EEG signal with a sample rate of 256Hz. Since there is an overlap of 128 samples between consecutive FFTs, 23 FFTs have to finish computation before 128 input data cycles to ensure real-time computation. This implies a minimum throughput of 46 FFTs/second.

6.1 Input buffer

The input EEG signal is streamed into the system and stored in an input buffer. The signal is streamed in a *time-major* format, *i.e.*, the samples corresponding to all the channels are streamed in order before going to the next time step. Appropriate counters are used to store the inputs in the input buffer. Once sufficient data is present, the data is streamed to the FFT core in a *channel-major* format, *i.e.*, all 256 samples from a single channel are streamed to the FFT core before switching to the next channel.

The input buffer is implemented as Block RAMs. The overall input buffer block utilizes 142 LUTs, 44 FFs, and 11.5 BRAMs.

6.2 FFT

The pipelined and *in-place* architectures detailed in Section 5.1 are the two implementations which are considered for the FFT core. Assuming only a single instance of the FFT processor, the minimum operating frequency is calculated. The pipelined architecture computes one FFT every 261 clock cycles, implying a minimum clock frequency of $\approx 12\text{kHz}$ to achieve a throughput of 46 FFTs/second. The *in-place* architecture requires 2088 clock cycles to compute one FFT. Therefore, the minimum operating frequency would be little greater than 96kHz.

To maintain uniformity across the results, the operating frequency is set to 100kHz for both the architectures while reporting power.

6.3 Magnitude

The output of the FFT core consists of real and imaginary components of the frequency content separately. The CNN, however, requires just the magnitude of the frequency components. Therefore, the magnitude block is implemented, consisting of two multipliers and an adder. The output of the block is truncated to 8-bits and streamed to the CNN.

6.4 CNN

The CNN is implemented in a layer-pipelined way. Each layer consists of an input memory where part of the input is stored, a processing element which computes the dot products, and the weights memory which stores the weights (see Figure 7). The exact structure of the three components varies from layer to layer. In each case, the design consideration is to achieve real-time operation with minimum hardware utilization.

Since the convolution in the first layer has a stride of 2, computation of (16×55) outputs should happen every second. Assuming a single processing element computes a (5×5) dot product in the first layer per clock cycle, 20, 240 clock cycles would be necessary for computation of the (16×55) output. Since the operating frequency of the design is set to 100 kHz, 20, 240 clock cycles corresponds to 0.2 seconds, which is less than the constraint of 1 second. This implies that one processing element is sufficient to compute the first convolutional layer in real-time. Similarly, for the second convolutional layer, output vector of shape (32×25) should be computed every 2 seconds. One processing element takes 12, 800 clock cycles for the computation of this output and therefore is sufficient to ensure real-time processing. Similarly, the third convolutional layer and the two fully-connected layers also requires one processing element each.

Table 6 shows the breakup of hardware utilization of the various blocks. The reported power is calculated by synthesizing each block separately, obtaining the power estimate from Vivado, and multiplying that with the fraction of time the block is active.

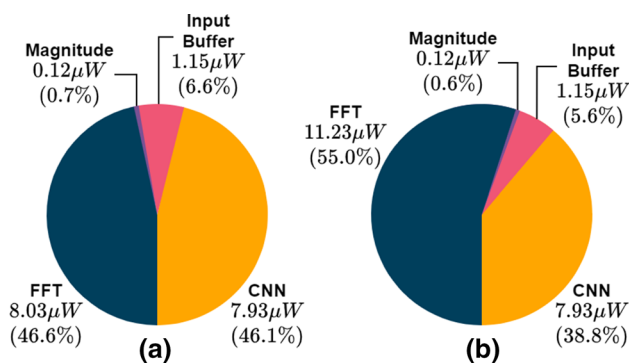


Fig. 9 Breakup of power consumed by various blocks in the prediction system when the FFT core is based on **a** pipelined architecture and **b** in-place architecture

The entire system, when implemented with the pipelined architecture, utilizes 4675 LUTs, 4296 FFs, 119 BRAMs, 57 DSP slices, while consuming $17.18\mu\text{W}$ of power. The in-place architecture utilizes 3913 LUTs, 3269 FFs, 109.5 BRAMs, 47 DSP slices, and consumes $20.39\mu\text{W}$ of power.

Figure 9 visually depicts the breakdown of the power consumed by the various blocks for both the pipelined and in-place based architectures. It is to be noted that although the device utilization is slightly lower for the in-place architecture when compared to the pipelined architecture, the power consumption of the pipelined architecture is lower as the pipelined architecture benefits from many stage-specific optimizations which are not feasible in the in-place architecture.

7 Conclusion

This paper implements and compares different hardware architectures for real-valued FFT algorithms. The architectures are based on a modified radix-2 FFT algorithm to utilize the fact that the Fourier transform of a real-valued signal is conjugate-symmetric. Single Processing Element (SPE), pipelined, and in-place variants of the algorithm are implemented and compared. We conclude that the in-place architecture requires the least number of device resources when compared to the pipelined architecture, while the throughput of the pipelined architecture is approximately 8 times that of the in-place architecture, for a 256-point FFT. These RFFT architectures are then used to generate feature vectors for a machine-learning based epileptic seizure prediction system. The pipelined implementation of the feature extraction in the seizure prediction system consumes $\approx 30\%$ less power when compared with the in-place implementation of feature extraction.

Data availability The data of simulation results used to support the findings of this study are included within the article.

References

- Rabiner, L. R., & Gold, B. (1975). *Theory and Application of Digital Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall.
- Ayinala, M., et al. (2013). An In-place FFT architecture for real-valued signals. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 60(10), 652–656.
- Yin, X., et al. (2016). Resource-efficient Pipelined architectures for radix-2 real-valued FFT with real datapaths. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 63(8), 803–807.
- Garrido, M., et al. (2009). A Pipelined FFT architecture for real-valued signals. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 56(12), 2634–2643.
- Meher, P. K., et al. (2015). Efficient VLSI architecture for decimation-in-time fast Fourier transform of real-valued data. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 62(12), 2836–2845.
- Mohanty, B. K., & Meher, P. K. (2019). Area-delay-energy efficient VLSI architecture for scalable in-place computation of FFT on real data. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 66(3), 1042–1050.
- Ayinala, M., et al. (2012). Pipelined parallel FFT architectures via folding transformation. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 20(6), 1068–1081.
- Salehi, S. A., et al. (2013). Pipelined architectures for real-valued FFT and Hermitian-symmetric IFFT with real datapaths. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 60(8), 507–511.
- Zhou, et al. (2018). Epileptic Seizure detection based on EEG signals and CNN. *Frontiers in Neuroinformatics*, 12, 1–14.
- Sanjeet, S., Sahoo, B. D., & Parhi, K. K. (2021). Comparison of real-valued FFT architectures for low-throughput applications using FPGA. In: *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 112–115.
- <https://www.epilepsy.com/learn/about-epilepsy-basics/epilepsy-statistics>
- Verma, N., Shoeb, A., Bohorquez, J., Dawson, J., Gutttag, J., & Chandrakasan, A. P. (2010). A micro-power EEG acquisition SoC with integrated feature extraction processor for a chronic seizure detection system. *IEEE Journal of Solid-State Circuits*, 45(4), 804–816.
- Zhang, Z., & Parhi, K. K. (2016). Low-complexity seizure prediction from iEEG/sEEG using spectral power and ratios of spectral power. *IEEE Transactions on Biomedical Circuits and Systems*, 10(3), 693–706.
- Park, Y., Luo, L., Parhi, K. K., & Netoff, T. (2011). Seizure prediction with spectral power of EEG using cost-sensitive support vector machines. *Epilepsia*, 52, 1761–1770.
- Bandarabadi, M., Dourado, A., Teixeira, C. A., Netoff, T. I., & Parhi, K. K. (2013). Seizure prediction with bipolar spectral power features using adaboost and SVM classifiers. In: *2013 35th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, Osaka, pp. 6305–6308.
- Zhang, Z., & Parhi, K. K. (2015). Seizure prediction using polynomial SVM classification. In: *2013 35th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, Milan, pp. 5748–5751.

17. Shiao, H., et al. (2017). SVM-based system for prediction of epileptic seizures from iEEG signal. *IEEE Transactions on Biomedical Engineering*, 64(5), 1011–1022.
18. Williamson, J. R., Bliss, D. W., Browne, D. W., & Narayanan, J. T. (2012). Seizure prediction using EEG spatiotemporal correlation structure. *Epilepsy & Behavior*, 25(2), 230–238.
19. Truong, N. D., Nguyen, A. D., Kuhlmann, L., Bonyadi, M. R., Yang, J., & Kavehei, O. (2017). A generalised seizure prediction with convolutional neural networks for intracranial and scalp electroencephalogram data analysis. [arXiv:1707.01976](https://arxiv.org/abs/1707.01976) [cs.CV].
20. Petrosian, A., Prokhorov, D., Homan, R., Dasheiff, R., & Wunsch, D. (2000). Recurrent neural network based prediction of epileptic seizures in intra- and extracranial EEG. *Neurocomputing*, 30(1–4), 201–218.
21. Daoud, H., & Bayoumi, M. A. (2019). Efficient epileptic seizure prediction based on deep learning. *IEEE Transactions on Biomedical Circuits and Systems*, 13(5), 804–813.
22. Truong, N. D., Nguyen, A. D., Kuhlmann, L., Bonyadi, M. R., Yang, J., Ippolito, S., & Kavehei, O. (2018). Convolutional neural networks for seizure prediction using intracranial and scalp electroencephalogram. *Neural Networks*, 105, 104–111.
23. Tsiouris, K. M., Pezoulas, V. C., Zervakis, M., Konitsiotis, S., Koutsouris, D. D., & Fotiadis, D. I. (2018). A long short-term memory deep learning network for the prediction of epileptic seizures using EEG signals. *Computers in Biology and Medicine*, 99, 24–37.
24. Khan, H., Marcuse, L., Fields, M., Swann, K., & Yener, B. (2018). Focal onset seizure prediction using convolutional networks. *IEEE Transactions on Biomedical Engineering*, 65(9), 2109–2118.
25. Mirowski, P. W., LeCun, Y., Madhavan, D., & Kuzniecky, R. (2008). Comparing SVM and convolutional networks for epileptic seizure prediction from intracranial EEG. In: *IEEE Workshop on Machine Learning for Signal Processing*, Cancun, pp. 244–249.
26. Wang, H., Shi, W., & Choy, C. (2017). Integrating channel selection and feature selection in a real time epileptic seizure detection system. In: *2017 39th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*. Seogwipo, pp. 3206–3211.
27. Chandler, D., Bisasky, J., Stanislaus, J.L.V.M., & Mohsenin, T. (2011). Real-time multi-channel seizure detection and analysis hardware. In: *2011 IEEE Biomedical Circuits and Systems Conference (BioCAS)*, San Diego, CA, pp. 41–44.
28. Bisasky, J., Chandler, D., & Mohsenin, T. (2012). A many-core platform implemented for multi-channel seizure detection. In: *2012 IEEE International Symposium on Circuits and Systems (ISCAS)*. Seoul, pp. 564–567.
29. Altaf, M.A.B., & Yoo, J. (2013). A 1.52 μ J/classification patient-specific seizure classification processor using linear SVM. In: *2013 IEEE International Symposium on Circuits and Systems (ISCAS)*, Beijing, pp. 849–852.
30. Altaf, M.A.B., Tillak, J., Kifle, Y., & Yoo, J. (2013). A 1.83 μ J/Classification Nonlinear Support-Vector-Machine-Based Patient-Specific Seizure Classification SoC. In: *2013 IEEE International Solid-State Circuits Conference Digest of Technical Papers*, San Francisco, CA, pp. 100–101.
31. Salam, M. T., Sawan, M., & Nguyen, D. K. (2011). A novel low-power-implantable epileptic seizure-onset detector. *IEEE Transactions on Biomedical Circuits and Systems*, 5(6), 568–578.
32. Shoeib, A.H. (2009). Application of machine learning to epileptic seizure onset detection and treatment, PhD diss., Massachusetts Institute of Technology.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Sai Sanjeet received the Dual Degree (B. Tech. and M. Tech.) in Electronics and Electrical Communication Engineering from the Indian Institute of Technology Kharagpur, Kharagpur, India, in 2021 with specialization in the area of Visual Image Processing and Embedded Systems. He is currently pursuing his PhD at Indian Institute of Technology Kharagpur, Kharagpur, India in the area of analog and mixed signal hardware accelerators for machine learning applications. His research

interests include machine learning, neuro inspired computing, analog and mixed signal circuit design, and signal processing. He was part of a collaborative group from the University of Tokyo which received 3rd place in the 30th International Workshop on Logic and Synthesis (IWLS) Design Contest 2021. He is also a recipient of the National Talent Search Examination (NTSE) fellowship and Kishore Vaigyanik Protsahan Yojana (KVPY) fellowship.



Bibhu Datta Sahoo received the B.Tech. degree in electrical engineering from the Indian Institute of Technology Kharagpur, Kharagpur, India, in 1998, the M.S.E.E. degree from the University of Minnesota, Minneapolis, MN, USA, in 2000, and the Ph.D.E.E. degree from the University of California, Los Angeles, CA, USA, in 2009. From 2000 to 2006, he was with DSP Microelectronics Group, Broadcom Corporation, Irvine, CA, USA, where he designed analog and digital integrated circuits for

signal-processing applications. From December 2008 to February 2010, he was with Maxlinear Inc., Carlsbad, CA, USA, where he was involved in designing integrated circuits for CMOS TV tuners. From March 2010 to November 2010, he was a Post-Doctoral Researcher with the University of California, Los Angeles, CA, USA. From December 2010 to December 2011, he was an Assistant Professor with the Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology Kharagpur, Kharagpur, India. From December 2011 to April 2015, he was an Associate Professor with the Department of Electronics and Communication Engineering, Amrita University, Amritapuri, India. From January 2016 to April 2017 he was on sabbatical from Amrita University and was a Research Scientist at University of Illinois at Urbana-Champaign. Since August 2017 he has been Associate Professor in the Department of Electronics and Electrical Communication Engineering at Indian Institute of Technology Kharagpur, Kharagpur, India. His research interests include data converters, signal processing, and analog and mixed signal circuit

design. He received the 2008 Analog Devices Outstanding Student Designer Award and was the co-recipient of the 2013 CICC Best Paper Award. He was the Associate Editor of IEEE Transactions on Circuits and Systems-II from August 2014 to December 2015. Since Dec. 2018 he has been the Associate Editor of IEEE Open Journal of Circuits and Systems.



Keshab K. Parhi (Fellow, IEEE) received the B.Tech. degree from IIT Kharagpur, Kharagpur, India, in 1982, the M.S.E.E. degree from the University of Pennsylvania, Philadelphia, PA, USA, in 1984, and the Ph.D. degree from the University of California at Berkeley, Berkeley, CA, USA, in 1988. He has been with the University of Minnesota, Minneapolis, MN, USA, since 1988, where he is currently a Distinguished McKnight University Professor and Edgar F. Johnson Professor with the

Department of Electrical and Computer Engineering. He has published more than 650 articles. He is the inventor or co-inventor of 31 patents. He has authored the textbook VLSI Digital Signal Processing Systems (New York, NY, USA: Wiley, 1999), and coedited the reference book Digital Signal Processing for Multimedia Systems (Boca Raton, FL, USA: CRC Press, 1999). His current research interests include the VLSI architecture design and implementation of signal processing, and

machine learning systems, neuromorphic computing, data-driven neuroscience and biomarkers for neuro-psychiatric disorders, hardware security, and molecular computing. Dr. Parhi was a recipient of numerous awards including the 2017 Mac Van Valkenburg Award, the 2012 Charles A. Desoer Technical Achievement Award, and the 1999 Golden Jubilee Medal, from the IEEE Circuits and Systems society, the 2013 Distinguished Alumnus Award from IIT Kharagpur, the 2013 Graduate/Professional Teaching Award from the University of Minnesota, the 2004 F. E. Terman Award from the American Society of Engineering Education, the 2003 IEEE Kiyo Tomiyasu Technical Field Award, and the 2001 IEEE W. R. G. Baker Prize Paper Award. He has served as the Technical Program Co-Chair for the 1995 IEEE VLSI Signal Processing Workshop and the 1996 Application Specific Systems, Architectures, and Processors Conference, and as the General Chair for the 2002 IEEE Workshop on Signal Processing Systems. He served as a Board of Governors Elected Member for the IEEE Circuits and Systems Society from 2005 to 2007. He has served on the Editorial Boards for the IEEE TRANSACTIONS ON IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS-I: Regular Papers and IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS-II: Express Briefs, the IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, the IEEE TRANSACTIONS ON SIGNAL PROCESSING, the IEEE SIGNAL PROCESSING LETTERS, and the IEEE Signal Processing Magazine, and served as the Editor-in-Chief for the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: Regular Papers from 2004 to 2005. He currently serves on the Editorial Board for the Journal of Signal Processing Systems (Springer). He was the Distinguished Lecturer of the IEEE Circuits and Systems Society from 1996 to 1998 and from 2019 to 2020.